

# SECURE APPLICATION PLATFORM

Defence-in-Depth AWS Security Implementation

## ISHIKA RAWLANI PROJECT REPORT

Date: 22 September 2026

AWS Region: Asia Pacific (Mumbai) — ap-south-1

|                      |                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project objective    | Secure a small application through independent identity, edge filtering, data protection, least-privilege access, and audit/monitoring controls.            |
| Primary AWS services | Amazon Cognito, AWS WAF, AWS KMS, AWS Secrets Manager, AWS IAM, IAM Access Analyzer, AWS CloudTrail, Amazon CloudWatch, AWS Lambda, and Amazon API Gateway. |
| Evidence set         | 43 supplied screenshots, arranged by project phase and explained individually.                                                                              |

Confidentiality: Evidence descriptions avoid reproducing passwords, TOTP secrets, access tokens, API-key values, and other credential material.

# 1. Executive Overview

This report documents the Secure Application Platform (defence-in-depth) implementation completed in AWS Mumbai. The security design separates controls so that identity, edge filtering, data protection, least-privilege access and audit telemetry provide independent barriers. The evidence includes positive and negative tests for authentication, authorization, WAF blocking, secret retrieval, KMS denial, IAM simulation and audit monitoring.

The implementation uses Amazon Cognito for user identity and MFA, AWS WAF for managed and rate-based edge protection, a customer-managed KMS key plus Secrets Manager for secret protection and rotation, dedicated IAM roles with Access Analyzer for least privilege, and CloudTrail/CloudWatch for audit and alerting. The live evidence is organized according to the six phases in the supplied capstone guide.

### 3. Requirements Coverage Matrix

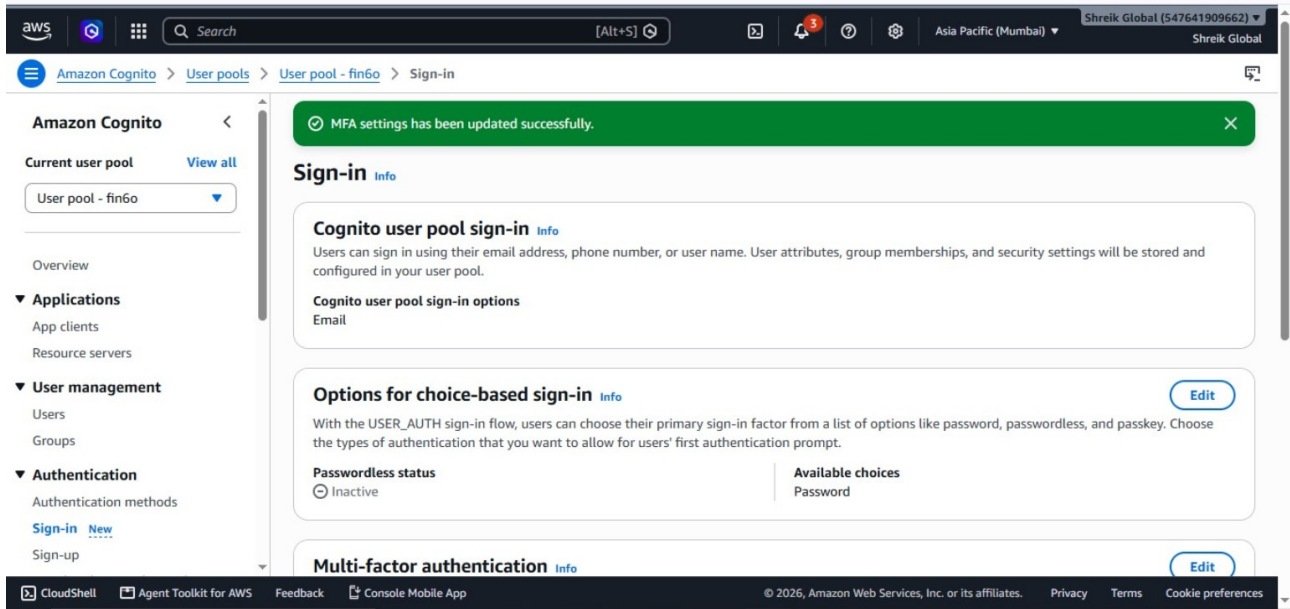
| Guide requirement       | Evidence / report coverage                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Phase 1 - Identity      | Cognito user pool, MFA/TOTP, groups, managed login, 401/200/403 authorization tests.                                        |
| Phase 2 - WAF           | Managed rules, rate-based rule, legitimate 200 response, XSS-like 403 response, WAF logs.                                   |
| Phase 3 - KMS + secrets | Customer-managed KMS key, Secrets Manager rotation, runtime retrieval, unauthorized secret/KMS denial.                      |
| Phase 4 - IAM           | Scoped runtime and rotation policies, Access Analyzer, allowed action, implicit deny, unrelated-secret blast-radius denial. |
| Phase 5 - Audit         | CloudTrail, WAF logs, denied decrypt evidence, WAF block evidence, CloudWatch alarms and triggered denied-decrypt alert.    |

## Phase 1 — Identity Layer (Amazon Cognito)

The identity layer establishes authentication and authorization boundaries using a Cognito user pool, managed login, MFA and group claims. The application validates bearer access tokens and restricts the admin route to members of the admin group.

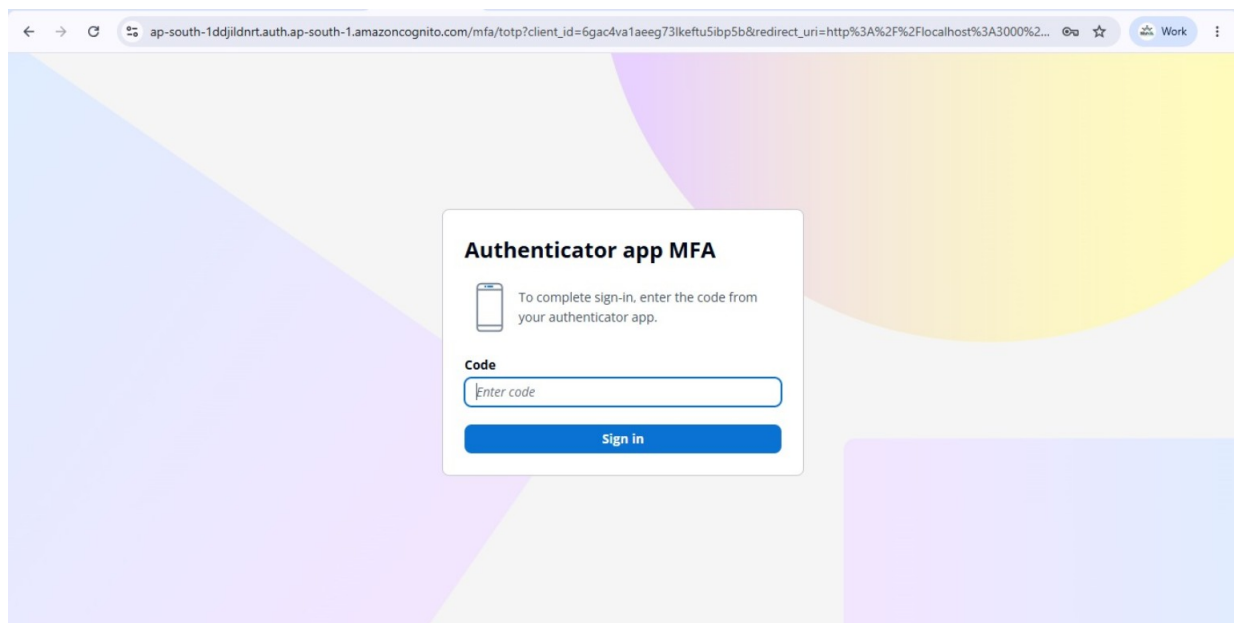
Outcome: Cognito MFA and group-based authorization were demonstrated with 401 unauthenticated, 200 authenticated-user, 403 non-admin, and 200 admin results.

Figure 58: Evidence screenshot



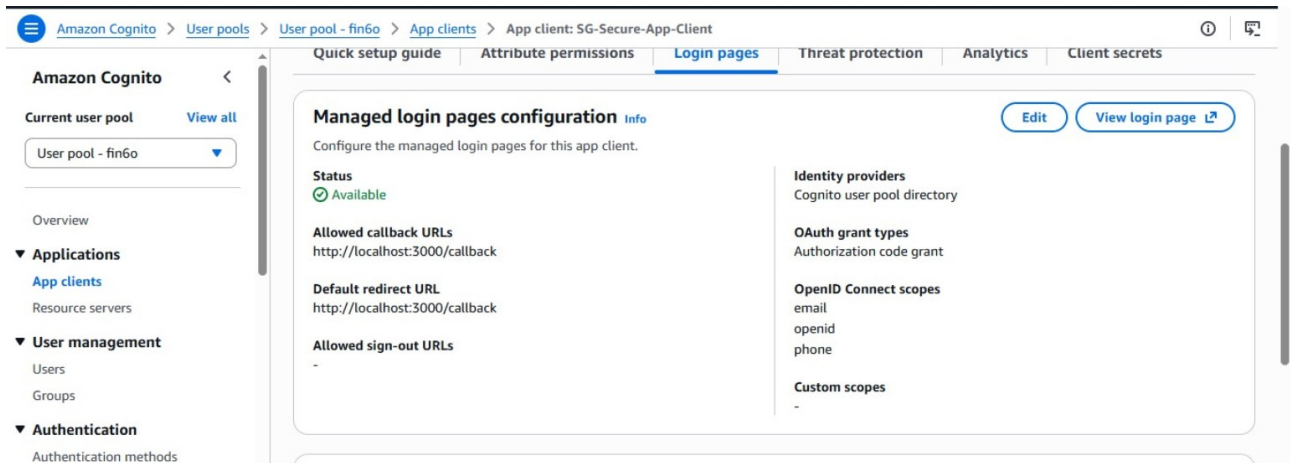
This screenshot confirms the Cognito user-pool sign-in configuration and successful MFA configuration update. Email is enabled as the sign-in identifier, while the multi-factor authentication configuration is visible in the same user-pool security context. It establishes the identity-layer baseline used for the protected API.

Figure 59: Evidence screenshot



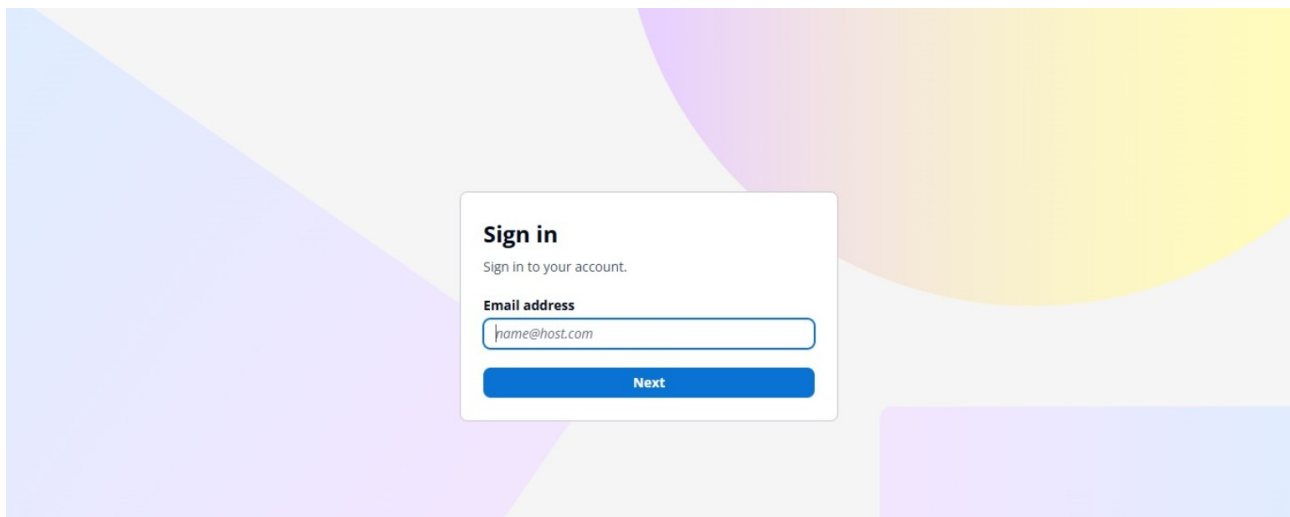
The managed Cognito login flow is displaying the authenticator-app MFA challenge. The screen demonstrates that a TOTP code is required as an additional authentication factor during sign-in. No secret seed or one-time code is reproduced in the report.

Figure 60: Evidence screenshot



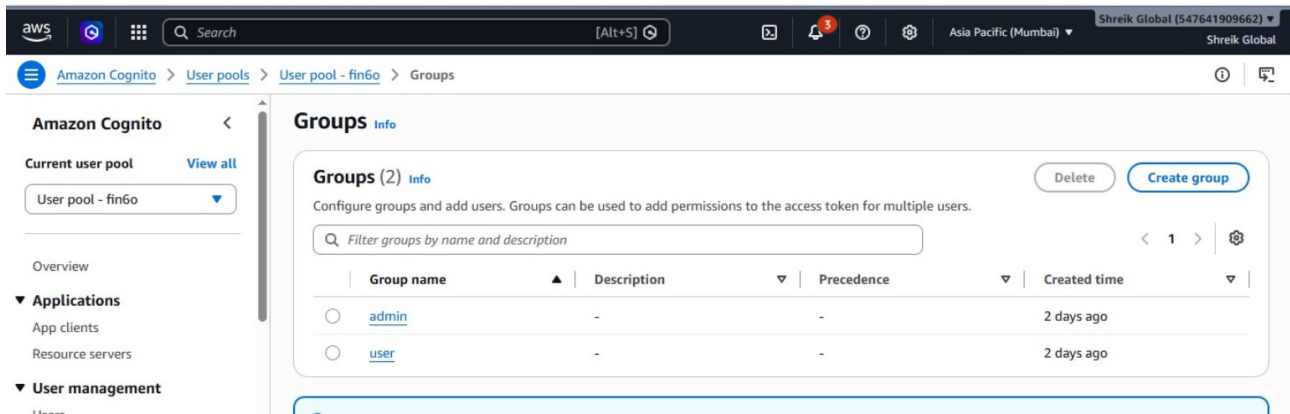
The Cognito app client configuration shows Managed Login as available, with the Cognito user-pool directory as the identity provider. The authorization-code OAuth flow and localhost callback are configured, together with OpenID Connect scopes. This provides the browser-based authentication entry point for the application.

Figure 61: Evidence screenshot



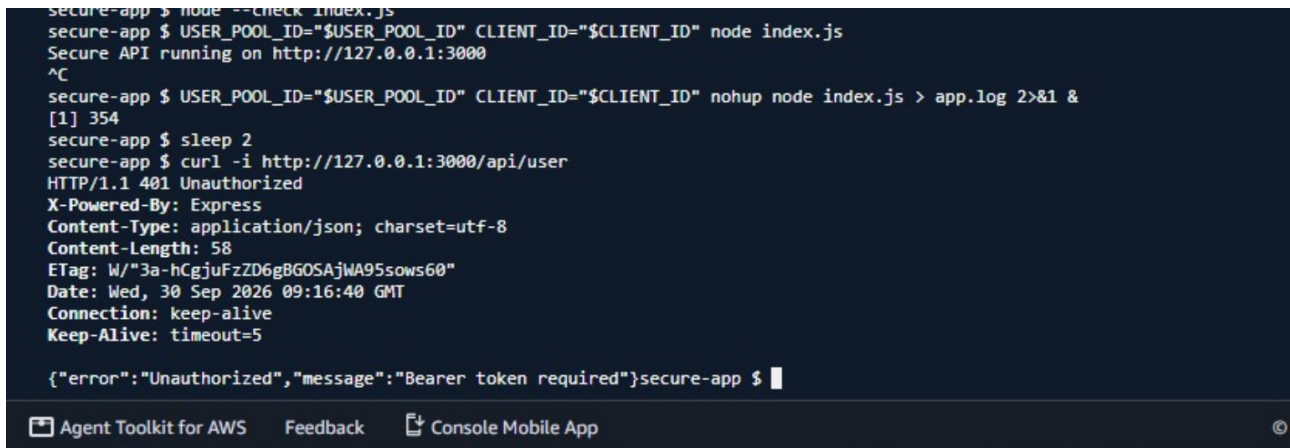
This screenshot shows the Cognito Managed Login sign-in page requesting the user's email address. It demonstrates the user-facing authentication entry point configured for the secure application. The flow proceeds to password and MFA validation before an authenticated session is established.

Figure 62: Evidence screenshot



The Cognito Groups page shows separate admin and user groups. Group membership is used by the application to distinguish ordinary authenticated access from the admin-only route. This provides the authorization boundary required by the project.

Figure 63: Evidence screenshot



The local Express API is started in CloudShell and an unauthenticated request to `/api/user` returns HTTP 401 Unauthorized with the message that a bearer token is required. This is the first direct proof that protected application access is denied before authentication.

**Figure 64: Evidence screenshot**

```
secure-app $ export VERIFY_RESPONSE=$(aws cognito-idp verify-software-token \
> --session "$VERIFY_SESSION" \
> --user-code "$MFA_CODE" \
> --friendly-device-name "SG-Secure-App-Authenticator" \
> --region "$AWS_REGION")
secure-app $ echo "$VERIFY_RESPONSE" | jq .{Status,Session}'
{
  "Status": "SUCCESS",
  "Session": "AYABeFbXQZUbaib6KntwVT0ZgqHA0BAedTXZ02wMlAB8B02duaXrVjVXl1c18vb2zaAEAF2F3cy1rbxM4TGfYb;phd3f6a212OmFwLXNvXRoLT6g;JUH0MDNMDQWkzkwOmtLe58yHjg1gWUJC05NTHzLTRNDctYjy4ly1hj3gwJyMyND
kx2Jk0uAECaQ84B1_cRcinunM71V8l1Tdf1K2qPhZSo0p-ctq9qjBP4t81S8Y9mK-I-U-T356uk19BBAaAAHwF4YJ3oZlhwNACq68wQ1BADBoBqkqh;1G9w8B8wHhVgYI2IZAHLOBAEwBEEDMDzGjvkr_J3NCNwC18E1A7zRatR-1A-14uGawta86dGc
yWk1fktmzyl4EYHf8Bd7suat-1ZK26_1BAQM0Q4TT_cY1EhctthjEVf0MJC9AAAAHwAAAAAAM4CvYQk1mCDGeymYaq13f_BAAABAAAAAABAAAAAAB55v0jv9v_ekz1FUTKc14syTQzVLL8_AgTlwtGzPc_-Zh2G3H3x-7Jf6-0
DP5SD5scsmoz_Tkkz127Q96R1KcY9qjPcSkorySbepJUKF07Kq6bF6VeXIDMf15xvb631G8sH7FEBUCd-m2BvjbyrYp4K1de-HyTun88R_FxMNC1Xp_pov1kxV378VcUc-gyFLGxbz1SU0SMbVj0_4DEzmzdz-alyMq6-1Gv3zHfvmQy1nig6epv6l
Qr2ElaR1x1nd-4tkBLP5mlwRpjncakTqSrY51388aIPHu-vsh1SeSdg5F12HJH3-ryG6aZe2K3QD4QT1BUJtr8vnto9G0VSpdV5_Dw7u02OnR14p4ktU7wz8272J3_wHfD2-hs13Wfgz1036GwTITVfmx473Uq335tFKnemuc_LAGy6wq6aveQGS1H6L
nqQG9_L3y0XQZc"
}
secure-app $
```

The Cognito software-token verification command returns Status=SUCCESS and a valid MFA setup session. The screenshot demonstrates that the authenticator-app TOTP factor was successfully verified. The raw session value is intentionally not reproduced in the report text.

**Figure 65: Evidence screenshot**

```
> --user-pool-id "$USER_POOL_ID" \  
> --client-id "$TEST_CLIENT_ID" \  
> --challenge-name MFA_SETUP \  
> --challenge-responses USERNAME=user@sgsecureapp.com \  
> --session "$(echo "$VERIFY_RESPONSE" | jq -r '.Session')" \  
> --region "$AWS_REGION")  
secure-app $  
secure-app $ echo "$FINAL_AUTH" | jq -r '  
> if .AuthenticationResult then  
>   "MFA authentication completed successfully"  
> else  
>   .  
> end'  
MFA authentication completed successfully
```

The subsequent Cognito challenge flow reports that MFA authentication completed successfully. This confirms the user can complete the required second factor and proceed to obtain an authenticated session.

Figure 67: Evidence screenshot

```

{"message":"Authenticated user access granted","username":"61237d1a-b071-70fe-fb8a-9490280aa449"}secure-app $
secure-app $ curl -i http://127.0.0.1:3000/api/admin \
> -H "Authorization: Bearer $ACCESS_TOKEN"
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 65
ETag: W/"41-sbqcfnfNzTAo8NkrRqqscayo06A"
Date: Wed, 30 Sep 2026 10:28:07 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Forbidden","message":"Admin group membership required"}secure-app $ █

```

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon V

The protected local Express API is tested with a valid bearer token and then the same authenticated non-admin context is used against /api/admin. The admin route returns HTTP 403 Forbidden with 'Admin group membership required', proving group-based authorization is enforced.

Figure 68: Evidence screenshot

```

ap-south-1 x ap-south-1 x +
> export ADMIN_FINAL=$(aws cognito-idp admin-respond-to-auth-challenge \
> --user-pool-id "$USER_POOL_ID" \
> --client-id "$TEST_CLIENT_ID" \
> --challenge-name MFA_SETUP \
> --challenge-responses "USERNAME=$(echo "$ADMIN_AUTH" | jq -r '.ChallengeParameters.USER_ID_FOR_SRP')" \
> --session "$(echo "$MFA_VERIFY" | jq -r '.Session')" \
> --region "$AWS_REGION") && \
>
> unset MFA_SECRET TOTP_CODE
secure-app $
secure-app $ echo "$MFA_VERIFY" | jq '{TOTP_Status:.Status,HasSetupSession:(.Session != null)}'
{
  "TOTP_Status": "SUCCESS",
  "HasSetupSession": true
}
secure-app $ echo "$ADMIN_FINAL" | jq '{AdminLoginSuccess:(.AuthenticationResult != null),HasAccessToken:(.AuthenticationResult.AccessToken != null)}'
{
  "AdminLoginSuccess": true,
  "HasAccessToken": true
}
secure-app $ export ADMIN_ACCESS_TOKEN=$(echo "$ADMIN_FINAL" | jq -r '.AuthenticationResult.AccessToken')
secure-app $ curl -i http://127.0.0.1:3000/api/admin \
> -H "Authorization: Bearer $ADMIN_ACCESS_TOKEN"
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 84
ETag: W/"54-q5QXhReojvvjKvbwcr8BkDmkuRg"
Date: Wed, 30 Sep 2026 11:17:25 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Admin access granted","username":"31936d2a-a0e1-708c-44a0-c8af89edd540"}secure-app $ █

```

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Term

A fresh admin MFA flow completes with TOTP status SUCCESS and an access token is obtained. The token is then used against /api/admin and the endpoint returns HTTP 200 OK with 'Admin access granted'. This completes the positive admin authorization proof.

## Phase 2 — Edge Protection (AWS WAF)

The WAF layer protects the API Gateway endpoint with managed rules and rate-based controls. Evidence covers both legitimate traffic and malicious-looking requests that receive HTTP 403 responses.

Outcome: WAF was associated with the API Gateway stage; legitimate traffic passed, XSS-like input was blocked, and rate/blocked-request testing was performed.

Figure 66: Evidence screenshot

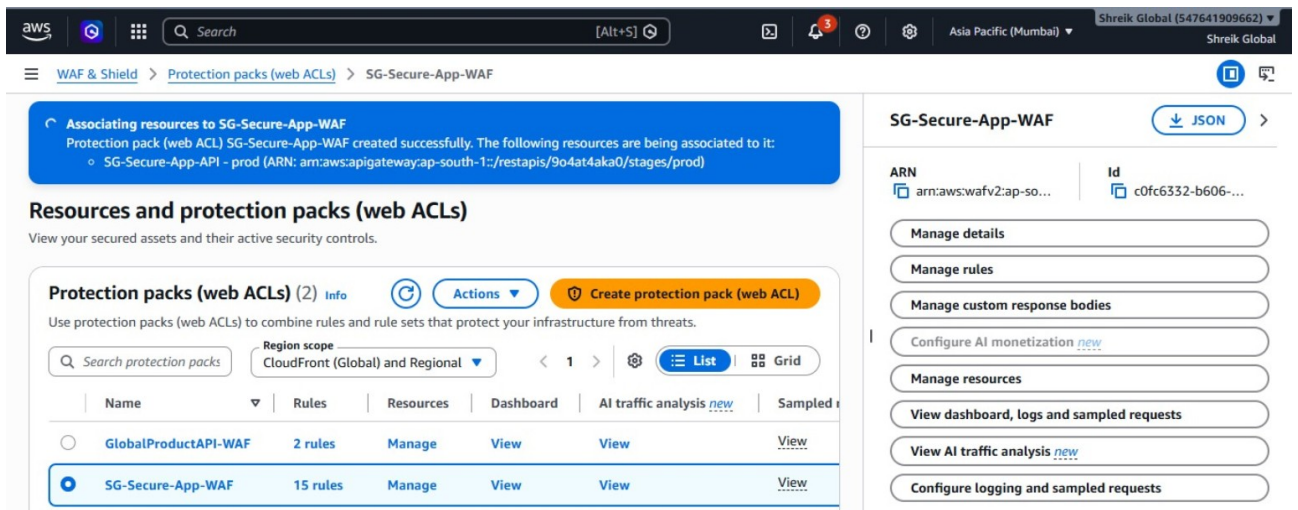
```
secure-app $
secure-app $ USER_POOL_ID="$USER_POOL_ID" CLIENT_ID="$CLIENT_ID" nohup node index.js > app.log 2>&1 &
[1] 1034
secure-app $ cat app.log
nohup: ignoring input
Secure API running on http://127.0.0.1:3000
secure-app $ curl -i http://127.0.0.1:3000/api/user \
> -H "Authorization: Bearer $ACCESS_TOKEN"
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 97
ETag: W/"61-QcL7ZRCpx+TRpymPTbTAb+IzFA4"
Date: Wed, 30 Sep 2026 10:20:24 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message": "Authenticated user access granted", "username": "61237d1a-b071-70fe-fb8a-9490280aa449"}
secure-app $
```

Agent Toolkit for AWS   Feedback   Console Mobile App   © 2026, Amazon Web Services, Inc. or its affiliates

The AWS WAF protection-pack screen shows SG-Secure-App-WAF associated with the SG-Secure-App-API production stage and configured with 15 rules. The web ACL includes managed and custom protections, providing the edge-security layer in front of the API Gateway test endpoint.

Figure 69: Evidence screenshot



The SG-Secure-App-WAF protection pack is shown in the AWS WAF console with 15 configured rules and the API Gateway production stage associated with it. This establishes the edge protection resource and its attachment to the intended API.

Figure 70: Evidence screenshot

```
secure-app $ curl -i "https://9o4at4aka0.execute-api.ap-south-1.amazonaws.com/prod/"
HTTP/2 200
date: Wed, 30 Sep 2026 11:47:39 GMT
content-type: application/json
content-length: 0
x-amzn-requestid: 8dbcd423-40a7-4603-ba21-029f7256e976
x-amz-apigw-id: EguCbHnyBcwEeJQ=
secure-app $
```

Agent Toolkit for AWS   Feedback   Console Mobile App   © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

A legitimate request to the WAF-protected API Gateway endpoint returns HTTP 200. This is the positive WAF test demonstrating that normal traffic can pass through the web ACL.

Figure 71: Evidence screenshot

```
secure-app $ curl -i "https://9o4at4aka0.execute-api.ap-south-1.amazonaws.com/prod/?q=%3Cscript%3Ealert(0)"
HTTP/2 403
date: Wed, 30 Sep 2026 11:48:55 GMT
content-type: application/json
content-length: 23
x-amzn-requestid: 8fd0db05-4d5d-4ca2-b85e-baf1b06b134c
x-amzn-errortype: ForbiddenException
x-amz-apigw-id: EguOUF7B8cwEOAg=
{"message":"Forbidden"}secure-app $
```

Agent Toolkit for AWS   Feedback   Console Mobile App

The WAF negative test sends an XSS-like query-string payload and receives HTTP 403 Forbidden. This demonstrates that the malicious-looking request is blocked at the WAF-protected API endpoint.

Figure 72: Evidence screenshot

```
secure-app $ curl -i "https://9o4at4aka0.execute-api.ap-south-1.amazonaws.com/prod/"
HTTP/2 403
date: Wed, 30 Sep 2026 11:55:16 GMT
content-type: application/json
content-length: 23
x-amzn-requestid: 68ebc2a0-e01e-4b99-88e2-0796ca0fe94d
x-amzn-errortype: ForbiddenException
x-amz-apigw-id: EgvJxGUw8cwEaXQ=

{"message":"Forbidden"}secure-app $
```

 Agent Toolkit for AWS    [Feedback](#)     Console Mobile App

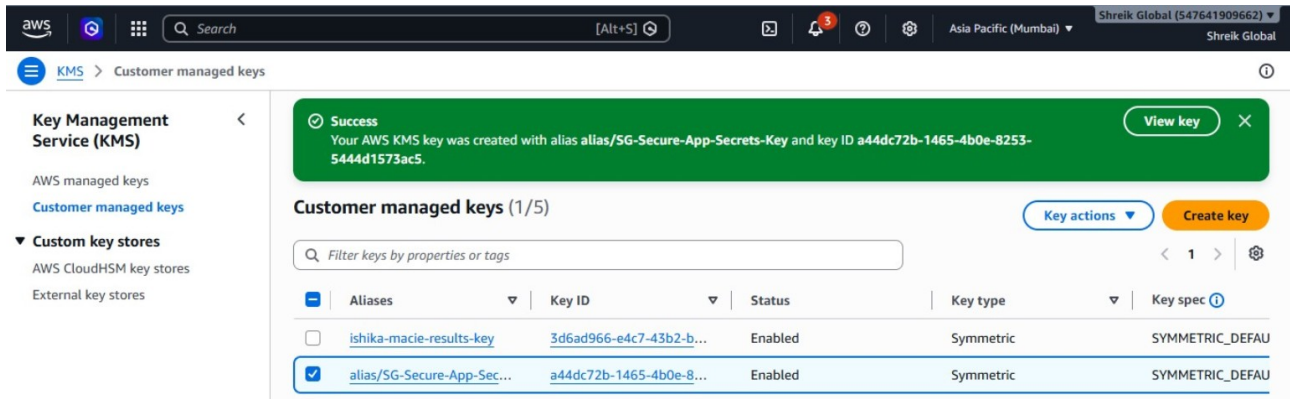
The additional WAF test output shows the blocked-response behavior after the security tests. It forms part of the rate/blocking validation evidence used alongside the configured rate-based rule and WAF metrics.

## Phase 3 — Data Protection (KMS + Secrets Manager)

The data-protection layer uses a customer-managed KMS key to protect a Secrets Manager secret, automatic Lambda-based rotation, runtime retrieval and explicit denial tests. CloudTrail provides the audit trail for secret and cryptographic operations.

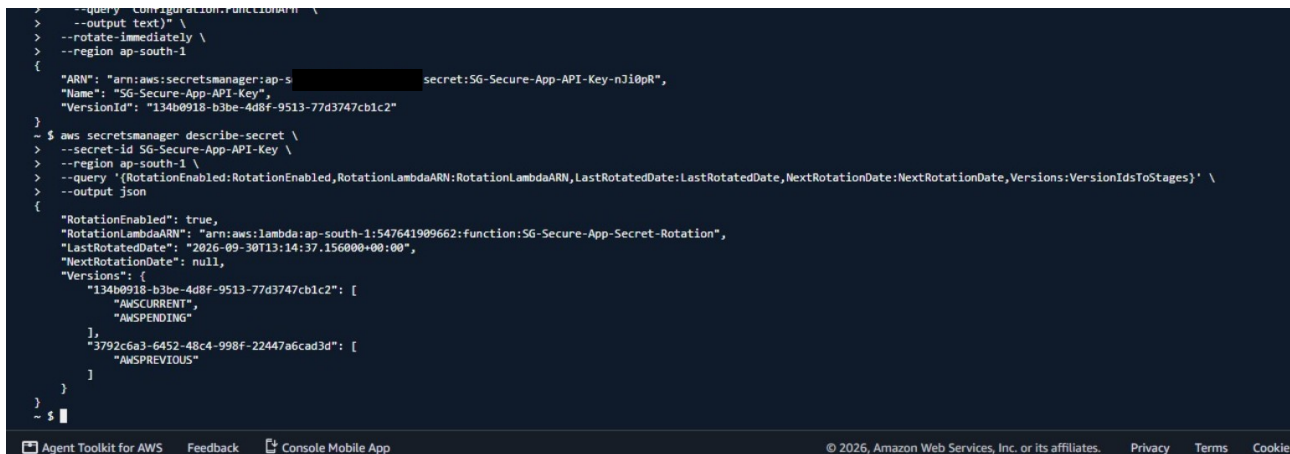
Outcome: Secrets were encrypted with a customer-managed KMS key, rotated through a Lambda workflow, read at runtime, and denied to unauthorized roles; TLS and CloudTrail evidence were collected.

Figure 73: Evidence screenshot



The KMS console confirms creation of the customer-managed symmetric key aliased as SG-Secure-App-Secrets-Key. The key is enabled and is used as the dedicated encryption key for the application secret. This establishes customer-managed encryption for the Secrets Manager data.

Figure 74: Evidence screenshot



The Secrets Manager CLI output shows SG-Secure-App-API-Key with automatic rotation enabled and the custom rotation Lambda configured. The version staging labels show AWSCURRENT/AWSPENDING on the active version and AWSPREVIOUS on the previous version, providing evidence of the rotation lifecycle.

Figure 75: Evidence screenshot

```
tmp $ aws lambda invoke \
> --function-name SG-Secure-App-Runtime \
> --region ap-south-1 \
> --payload '{}' \
> /tmp/runtime-result.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
tmp $ cat /tmp/runtime-result.json
{"statusCode": 200, "secret_loaded": true, "message": "Secret retrieved successfully at runtime"}tmp $
```

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Priv

The runtime Lambda invocation returns statusCode 200 with secret\_loaded=true and the message that the secret was retrieved successfully at runtime. This proves that the application component obtains the secret through Secrets Manager at runtime rather than embedding its value in source code.

Figure 76: Evidence screenshot

```
ap-south-1 +
"x86_64"
},
"EphemeralStorage": {
  "Size": 512
},
"SnapStart": {
  "ApplyOn": "None",
  "OptimizationStatus": "Off"
},
tmp $ aws lambda get-function \
> --function-name SG-Secure-App-Unauthorized-Test \
> --region ap-south-1 \
> --query 'Configuration.State' \
> --output text
Active
tmp $ aws lambda invoke \
> --function-name SG-Secure-App-Unauthorized-Test \
> --region ap-south-1 \
> --payload '{}' \
> /tmp/unauthorized-result.json
{
  "StatusCode": 200,
  "FunctionError": "Unhandled",
  "ExecutedVersion": "$LATEST"
}
tmp $ cat /tmp/unauthorized-result.json
{"errorMessage": "An error occurred (AccessDeniedException) when calling the GetSecretValue operation: User: arn:aws:sts::62:assumed-role/SG-Secure-App-Unauthorized-Role/SG-Secure-App-Unauthorized-Test is not authorized to perform: secretsmanager:GetSecretValue on resource: SG-Secure-App-API-Key because no identity-based policy allows the secretsmanager:GetSecretValue action", "errorType": "ClientError", "requestId": "ca00c497-18f3-499a-8f79-817c9ee17f06", "stackTrace": [" File \"/var/task/unauthorized_lambda.py", line 6, in lambda_handler\n response = secretsmanager.get_secret_value(\n, " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 686, in _api_call\n return self._make_api_call(operation_name, kwargs)\n, " File \"/var/lang/lib/python3.12/site-packages/botocore/context.py", line 123, in wrapper\n return func(*args, **kwargs)\n, " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 1094, in _make_api_call\n raise error_class(parsed_response, operation_name)\n"]}tmp $
```

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

The unauthorized Lambda test produces an AccessDeniedException for secretsmanager:GetSecretValue. The error identifies the unauthorized execution role and the protected secret resource. This is direct evidence that the secret cannot be read by an unapproved component.

Figure 77: Evidence screenshot

```

> --output json
{
  "Events": [
    {
      "EventId": "e48a0772-d3e2-4111-8d60-e9c6a18d4e47",
      "EventName": "GetSecretValue",
      "ReadOnly": "true",
      "AccessKeyId": "ASIAK7AQWGP645256N2",
      "EventTime": "2026-09-30T13:31:58+00:00",
      "EventSource": "secretsmanager.amazonaws.com",
      "UserName": "SG-Secure-App-Unauthorized-Test",
      "Resources": [],
      "CloudTrailEvent": "{\n  \"eventVersion\": \"1.11\", \"userIdentity\": {\n    \"type\": \"AssumedRole\", \"principalId\": \"AROAX7AQWGP6G0A6FAVZ:SG-Secure-App-Unauthorized-Test\", \"arn\": \"arn:aws:sts::547641909662:assumed-role/SG-Secure-App-Unauthorized-Test/SG-Secure-App-Unauthorized-Test\", \"accountId\": \"547641909662\", \"roleArn\": \"arn:aws:iam::547641909662:role/SG-Secure-App-Unauthorized-Role\", \"accountId\": \"547641909662\", \"roleName\": \"SG-Secure-App-Unauthorized-Role\", \"attributes\": {\n      \"creationDate\": \"2026-09-30T13:31:58Z\", \"mfaAuthenticated\": \"false\", \"inScopeOf\": {\n        \"issuerType\": \"AWS::Lambda::function\", \"credentialsIssuedTo\": \"arn:aws:lambda:ap-south-1:547641909662:function:SG-Secure-App-Unauthorized-Test\", \"credentialIssuedTo\": \"arn:aws:lambda:ap-south-1:547641909662:function:SG-Secure-App-Unauthorized-Test\", \"sourceIPAddress\": \"13.201.5.243\", \"eventTime\": \"2026-09-30T13:31:58Z\", \"eventSource\": \"secretsmanager.amazonaws.com\", \"eventName\": \"GetSecretValue\", \"awsRegion\": \"ap-south-1\", \"sourceIPAddress\": \"13.201.5.243\", \"userAgent\": \"Boto3/1.42.97 md/Botocore/1.42.97 ua/2.1 os/linux#5.10.255-268-309.1092.x86_64 md/arch#x86_64 lang/python#3.12.14 md/pyimpl#CPython exec-env/AWS_Lambda_python3.12 m/g,b,D,Z ctf/retry-modeLegacy Botocore/1.42.97\", \"errorCode\": \"AccessDenied\", \"errorMessage\": \"User: arn:aws:sts::547641909662:assumed-role/SG-Secure-App-Unauthorized-Role/SG-Secure-App-Unauthorized-Test is not authorized to perform: secretsmanager:GetSecretValue on resource: SG-Secure-App-API-Key because no identity-based policy allows the secretsmanager:GetSecretValue action\", \"requestParameters\": null, \"responseElements\": null, \"requestID\": \"7e2a94a3-bbc5-4922-baec-8664884330bb\", \"eventID\": \"e48a0772-d3e2-4111-8d60-e9c6a18d4e47\", \"readOnly\": true, \"eventType\": \"AwsApiCall\", \"managementEvent\": true, \"recipientAccountID\": \"547641909662\", \"eventCategory\": \"Management\", \"isDetails\": {\n        \"tlsVersion\": \"TLSv1.3\", \"cipherSuite\": \"TLS_AES_128_GCM_SHA256\", \"clientProvidedHostHeader\": \"secretsmanager.ap-south-1.amazonaws.com\", \"keyExchange\": \"x25519\"}\n    }\n  }\n}",
      "EventId": "d888af60-de2a-4009-9592-86e7aa48496c",
      "EventName": "GetSecretValue",
      "ReadOnly": "true",
      "AccessKeyId": "ASIAK7AQWGP645256N2",
      "EventTime": "2026-09-30T13:25:27+00:00",
      "EventSource": "secretsmanager.amazonaws.com",
    }
  ]
}

```

CloudTrail event evidence records a GetSecretValue activity associated with the unauthorized test role and shows the access-denied result in the event record. This provides the audit trail linking the attempted secret access to the calling identity.

Figure 78: Evidence screenshot

```

> --function-name SG-Secure-App-KMS-Unauthorized-Test \
> --region ap-south-1 \
> --query Configuration.State \
> --output text
Active
tmp $ CIPHERTEXT=$(cat /tmp/test-ciphertext.txt)
tmp $
tmp $ printf '{"ciphertext":"%s"}' "$CIPHERTEXT" > /tmp/kms-event.json
tmp $
tmp $ aws lambda invoke \
> --function-name SG-Secure-App-KMS-Unauthorized-Test \
> --region ap-south-1 \
> --payload fileb:///tmp/kms-event.json \
> /tmp/kms-denied-result.json
{
  "statusCode": 200,
  "functionError": "Unhandled",
  "executedVersion": "$LATEST"
}
tmp $ cat /tmp/kms-denied-result.json
{"errorMessage": "An error occurred (AccessDeniedException) when calling the Decrypt operation: User: arn:aws:sts::547641909662:assumed-role/SG-Secure-App-Unauthorized-Role/SG-Secure-App-KMS-Unauthorized-Test is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-south-1:547641909662:key/a4dc72b-1460-4b0e-8253-5444d1573ac5 because no identity-based policy allows the kms:Decrypt action", "errorType": "ClientError", "requestId": "0fae01c8-062a-4059-8cb7-2855a2195c60", "stackTrace": [" File \"/var/task/kms_denied_lambda.py", line 9, in lambda_handler\n  response = kms.decrypt(n", " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 606, in _api_call\n  return self._make_api_call(operation_name, kwargs)\n", " File \"/var/lang/lib/python3.12/site-packages/botocore/context.py", line 123, in wrapper\n  return func(*args, **kwargs)\n", " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 1094, in _make_api_call\n  raise error_class(parsed_response, operation_name)\n"]tmp $

```

The KMS unauthorized-decrypt test invokes the dedicated negative-test Lambda and receives an AccessDeniedException for kms:Decrypt. This demonstrates that possession of an execution role does not automatically grant cryptographic access to the customer-managed key.

Figure 79: Evidence screenshot

```
tmp $ curl -iv "https://9o4at4aka0.execute-api.ap-south-1.amazonaws.com/prod/" 2>&1 | grep -E "SSL connection|TLS|HTTP/"
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_128_GCM_SHA256 / x25519 / RSASSA-PSS
0 0 0 0 0 0* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://9o4at4aka0.execute-api.ap-south-1.amazonaws.com/prod/
* [HTTP/2] [1] [:method: HEAD]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: 9o4at4aka0.execute-api.ap-south-1.amazonaws.com]
* [HTTP/2] [1] [:path: /prod/]
* [HTTP/2] [1] [user-agent: curl/8.21.0]
* [HTTP/2] [1] [accept: */*]
> HEAD /prod/ HTTP/2
* TLSv1.3 (IN), TLS handshake, NewSession Ticket (4):
< HTTP/2 403
HTTP/2 403
tmp $
```

The TLS diagnostic shows a successful TLS 1.3 handshake and HTTP/2 connection to the API endpoint. The cipher suite and TLS version demonstrate strong transport encryption for the API path, even though the request response shown was blocked by the WAF.

Figure 80: Evidence screenshot

```
aws CloudShell
ap-south-1 +
GetSecretValue 2026-09-30T13:54:21+00:00 SG-Secure-App-Secret-Rotation
GetSecretValue 2026-09-30T13:31:58+00:00 SG-Secure-App-Unauthorized-Test
GetSecretValue 2026-09-30T13:25:27+00:00 SG-Secure-App-RunTime
GetSecretValue 2026-09-30T13:14:37+00:00 SG-Secure-App-Secret-Rotation
GetSecretValue 2026-09-30T13:14:36+00:00 SG-Secure-App-Secret-Rotation
GetSecretValue 2026-09-30T13:08:21+00:00 SecretsManagerstage3-RDS-Rotation-Lambda-Role
GetSecretValue 2026-09-30T13:08:20+00:00 SecretsManagerstage3-RDS-Rotation-Lambda-Role
GetSecretValue 2026-09-30T13:08:18+00:00 SecretsManagerstage3-RDS-Rotation-Lambda-Role
GetSecretValue 2026-09-30T13:08:18+00:00 SecretsManagerstage3-RDS-Rotation-Lambda-Role
tmp $ aws cloudtrail lookup-events \
> --lookup-attributes AttributeKey=EventName,AttributeValue=Decrypt \
> --max-results 10 \
> --region ap-south-1 \
> --query 'Events[].[Time:EventTime,User:Username,Event:EventName]' \
> --output table
+-----+-----+-----+
| Event | Time | User |
+-----+-----+-----+
| Decrypt | 2026-09-30T13:54:21+00:00 | SG-Secure-App-Secret-Rotation |
| Decrypt | 2026-09-30T13:44:06+00:00 | SG-Secure-App-KMS-Unauthorized-Test |
| Decrypt | 2026-09-30T13:42:48+00:00 | AmazonSecurityLakeMetastoreManager-ap-south-1 |
| Decrypt | 2026-09-30T13:28:54+00:00 | Inspector2LambdaScanning-39532744906683334 |
| Decrypt | 2026-09-30T13:28:54+00:00 | Inspector2LambdaScanning-39532744906683334 |
| Decrypt | 2026-09-30T13:28:54+00:00 | Inspector2LambdaScanning-39532744906683334 |
| Decrypt | 2026-09-30T13:28:54+00:00 | Inspector2LambdaScanning-39532744906683334 |
| Decrypt | 2026-09-30T13:28:54+00:00 | Inspector2LambdaScanning-39532744906683334 |
| Decrypt | 2026-09-30T13:28:54+00:00 | Inspector2LambdaScanning-39532744906683334 |
+-----+-----+-----+
tmp $
```

CloudTrail lookup evidence lists recent Decrypt and GetSecretValue activity, including the project rotation and runtime components and the unauthorized KMS test. This provides the requested 'who accessed what and when' audit evidence for the data-protection layer.

## Phase 4 — Least-Privilege IAM

Each application component uses a dedicated execution role with resource-scoped permissions. IAM policy simulation and Access Analyzer evidence demonstrate permitted operations, implicit denies and a constrained blast radius.

Outcome: Component-specific IAM roles were scoped to exact resources; unnecessary actions and unrelated secret access were denied; Access Analyzer was active.

Figure 81: Evidence screenshot

```
tmp $ aws accessanalyzer get-analyzer \
> --analyzer-name SG-Secure-App-Analyzer \
> --region ap-south-1 \
> --query 'analyzer.{Name:name,Status:status,Type:type}' \
> --output table
```

| Name                   | Status | Type    |
|------------------------|--------|---------|
| SG-Secure-App-Analyzer | ACTIVE | ACCOUNT |

```
tmp $
```

Agent Toolkit for AWS   Feedback   Console Mobile App

IAM Access Analyzer is shown in ACTIVE state as an account-level analyzer named SG-Secure-App-Analyzer. This establishes continuous analysis of access relationships for the security project and supports the least-privilege review process.

Figure 82: Evidence screenshot

```
CloudShell
ap-south-1 +
```

```
tmp $ aws iam get-role-policy \
> --role-name SG-Secure-App-RunTime-Role \
> --policy-name SG-Secure-App-Secret-Read \
> --query 'PolicyDocument.Statement' \
> --output json
```

```
[
  {
    "Sid": "ReadOnlyThisSecret",
    "Effect": "Allow",
    "Action": [
      "secretsmanager:GetSecretValue"
    ],
    "Resource": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:SG-Secure-App-API-Key-nJi8pR"
  },
  {
    "Sid": "DecryptOnlyThisSecret",
    "Effect": "Allow",
    "Action": [
      "kms:Decrypt",
      "kms:DescribeKey"
    ],
    "Resource": "arn:aws:kms:ap-south-1:123456789012:key/ac5",
    "Condition": {
      "StringEquals": {
        "kms:EncryptionContext:SecretARN": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:SG-Secure-App-API-Key-nJi8pR"
      }
    }
  }
]
```

```
tmp $
```

Agent Toolkit for AWS   Feedback   Console Mobile App   © 2026, Amazon Web Services, Inc. or its affiliates.   Privacy   Terms

The runtime execution-role policy grants secretsmanager:GetSecretValue only for the specific application secret. KMS decrypt/describe permissions are likewise scoped to the dedicated key and constrained with the SecretARN encryption context. This is a narrowly scoped service role rather than a broad administrator policy.

Figure 83: Evidence screenshot

```

tmp $ aws iam simulate-principal-policy \
> --policy-source-arn arn:aws:iam::[REDACTED] \
> --action-names s3:ListAllMyBuckets \
> --query 'EvaluationResults[0].{Action:EvalActionName,Decision:EvalDecision}' \
> --output table
-----
|      SimulatePrincipalPolicy      |
+-----+-----+
|      Action      |      Decision      |
+-----+-----+
| s3:ListAllMyBuckets | implicitDeny |
+-----+-----+

tmp $ aws iam simulate-principal-policy \
> --policy-source-arn arn:aws:iam::54[REDACTED]:role/SG-Secure-App-Runtime-Role \
> --action-names secretsmanager:GetSecretValue \
> --resource-arns "$(aws secretsmanager describe-secret \
> --secret-id SG-Secure-App-API-Key \
> --region ap-south-1 \
> --query ARN --output text)" \
> --query 'EvaluationResults[0].{Action:EvalActionName,Decision:EvalDecision}' \
> --output table
-----
|      SimulatePrincipalPolicy      |
+-----+-----+
|      Action      |      Decision      |
+-----+-----+
| secretsmanager:GetSecretValue | allowed |
+-----+-----+

tmp $

```

IAM policy simulation shows s3:ListAllMyBuckets receiving an implicitDeny for the runtime role. The test demonstrates that an unrelated service action is not available to the application component. This is direct evidence for least-privilege enforcement.

Figure 84: Evidence screenshot

```

tmp $ aws accessanalyzer list-findings-v2 \
> --analyzer-arn arn:aws:access-analyzer:ap-s[REDACTED]:[REDACTED]/SG-Secure-App-Analyzer \
> --region ap-south-1 \
> --query 'findings[0].id' \
> --output text
e7376a98-74b7-4a6e-8efd-ae8fcd23098d
tmp $ aws accessanalyzer get-finding-v2 \
> --analyzer-arn arn:aws:access-analyzer:ap-s[REDACTED]:[REDACTED]/SG-Secure-App-Analyzer \
> --id e7376a98-74b7-4a6e-8efd-ae8fcd23098d \
> --region ap-south-1 \
> --query '{Status:status,Resource:resource,Principal:principal,Action:action,Condition:condition}' \
> --output json
{
  "Status": "ACTIVE",
  "Resource": "arn:aws:iam::[REDACTED]:role/GitHubActions-GlobalProductAPI-DeployRole",
  "Principal": null,
  "Action": null,
  "Condition": null
}
tmp $

```

IAM policy simulation shows secretsmanager:GetSecretValue is allowed for the intended application secret. The permitted action test confirms that the scoped runtime role retains exactly the access needed for its function.

Figure 85: Evidence screenshot

```

> --role-name SG-Secure-App-Secret-Rotation-role-24je0mjb \
> --policy-name SG-Secure-App-Rotation-Access \
> --query 'PolicyDocument.Statement' \
> --output json
[
  {
    "Sid": "SecretRotationAccess",
    "Effect": "Allow",
    "Action": [
      "secretsmanager:DescribeSecret",
      "secretsmanager:GetSecretValue",
      "secretsmanager:PutSecretValue",
      "secretsmanager:UpdateSecretVersionStage"
    ],
    "Resource": "arn:aws:secretsmanager:ap-south-1:547641909662:secret:SG-Secure-App-API-Key-n3i0pR"
  },
  {
    "Sid": "KMSAccessForThisSecret",
    "Effect": "Allow",
    "Action": [
      "kms:Decrypt",
      "kms:DescribeKey",
      "kms:GenerateDataKey"
    ],
    "Resource": "arn:aws:kms:ap-south-1:547641909662:key/a44dc72b-1465-4b0e-8253-5444d1573ac5",
    "Condition": {
      "StringEquals": {
        "kms:EncryptionContext:SecretARN": "arn:aws:secretsmanager:ap-south-1:547641909662:secret:SG-Secure-App-API-Key-n3i0pR"
      }
    }
  }
]
tmp $

```

The rotation execution-role policy scopes Secrets Manager actions to the application secret and KMS actions to the dedicated customer-managed key. The encryption-context condition further binds cryptographic use to the intended secret, reducing the role's potential blast radius.

Figure 86: Evidence screenshot

```

> --output table
+-----+-----+
| SimulatePrincipalPolicy |
+-----+-----+
| Action | Decision |
+-----+-----+
| s3:ListAllMyBuckets | implicitDeny |
+-----+-----+
tmp $ aws lambda get-function-configuration \
> --function-name SG-Secure-App-Runtime \
> --region ap-south-1 \
> --query '(Function:FunctionName,ExecutionRole:Role)' \
> --output table
+-----+-----+
| GetFunctionConfiguration |
+-----+-----+
| ExecutionRole | Function |
+-----+-----+
| arn:aws:iam::547641909662:role/SG-Secure-App-Runtime-Role | SG-Secure-App-Runtime |
+-----+-----+
tmp $ aws lambda get-function-configuration \
> --function-name SG-Secure-App-Secret-Rotation \
> --region ap-south-1 \
> --query '(Function:FunctionName,ExecutionRole:Role)' \
> --output table
+-----+-----+
| GetFunctionConfiguration |
+-----+-----+
| ExecutionRole | Function |
+-----+-----+
| arn:aws:iam::547641909662:role/service-role/SG-Secure-App-Secret-Rotation-role-24je0mjb | SG-Secure-App-Secret-Rotation |
+-----+-----+
tmp $

```

The Lambda execution-role mapping shows SG-Secure-App-Runtime using SG-Secure-App-Runtime-Role and SG-Secure-App-Secret-Rotation using its dedicated rotation role. The separation demonstrates component-specific IAM identities instead of sharing one broad role.

Figure 87: Evidence screenshot

```
tmp $ aws accessanalyzer list-findings-v2 \
> --analyzer-arn arn:aws:access-analyzer:ap-south-1:██████████:analyzer/SG-Secure-App-Analyzer \
> --region ap-south-1 \
> --query 'findings[?contains(resource, 'SG-Secure-App')].{Status:status,Resource:resource,Type:findingType}' \
> --output table
tmp $ aws iam simulate-principal-policy \
> --policy-source-arn arn:aws:iam:5██████████ \
> --action-names secretsmanager:GetSecretValue \
> --resource-arns arn:aws:secretsmanager:ap-south-1:54██████████:secret:SG-Secure-App-OTHER-SECRET \
> --query 'EvaluationResults[.]{Action:EvalActionName,Resource:EvalResourceName,Decision:EvalDecision}' \
> --output table
```

| SimulatePrincipalPolicy       |              |                                                                                  |
|-------------------------------|--------------|----------------------------------------------------------------------------------|
| Action                        | Decision     | Resource                                                                         |
| secretsmanager:GetSecretValue | implicitDeny | arn:aws:secretsmanager:ap-south-1:547641909662:secret:SG-Secure-App-OTHER-SECRET |

tmp \$

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates.

The Access Analyzer finding output shows an existing account-level finding for an unrelated GitHubActions-GlobalProductAPI-DeployRole. This finding is outside the SG-Secure-App project roles and is therefore documented rather than incorrectly represented as a project-role failure.

Figure 88: Evidence screenshot

```
tmp $ aws cloudtrail Lookup-Events \
> --lookup-attributes AttributeKey=EventName,AttributeValue=GetSecretValue \
> --max-results 20 \
> --region ap-south-1 \
> --query 'Events[.]{Time:EventTime,User:Username,Event:EventName}' \
> --output table
```

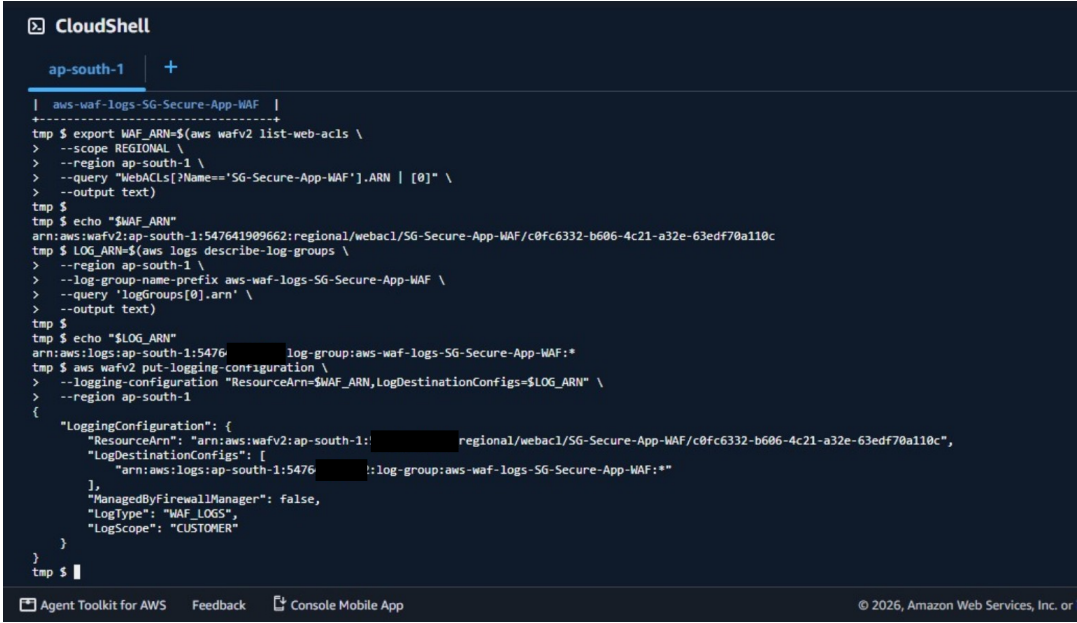
| LookupEvents   |                           |                                               |
|----------------|---------------------------|-----------------------------------------------|
| Event          | Time                      | User                                          |
| GetSecretValue | 2026-09-30T13:54:21+00:00 | SG-Secure-App-Secret-Rotation                 |
| GetSecretValue | 2026-09-30T13:54:21+00:00 | SG-Secure-App-Secret-Rotation                 |
| GetSecretValue | 2026-09-30T13:31:58+00:00 | SG-Secure-App-Unauthorized-Test               |
| GetSecretValue | 2026-09-30T13:25:27+00:00 | SG-Secure-App-RunTime                         |
| GetSecretValue | 2026-09-30T13:14:37+00:00 | SG-Secure-App-Secret-Rotation                 |
| GetSecretValue | 2026-09-30T13:14:36+00:00 | SG-Secure-App-Secret-Rotation                 |
| GetSecretValue | 2026-09-30T13:08:21+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T13:08:20+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T13:08:18+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T13:08:18+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T13:08:17+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T13:08:17+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T13:07:59+00:00 | stage3-api-key-rotation                       |
| GetSecretValue | 2026-09-30T13:07:59+00:00 | stage3-api-key-rotation                       |
| GetSecretValue | 2026-09-30T13:07:59+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T09:08:02+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T09:08:01+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T09:07:59+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T09:07:59+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |
| GetSecretValue | 2026-09-30T09:07:59+00:00 | SecretsManagerstage3-RDS-Rotation-Lambda-Role |

tmp \$

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates.

The IAM simulation demonstrates that attempting to read a different secret, SG-Secure-App-OTHER-SECRET, results in implicitDeny. This is the project's blast-radius proof: compromising the runtime role does not automatically grant access to unrelated secrets.

Figure 89: Evidence screenshot



```
CloudShell
ap-south-1 +
| aws-waf-logs-SG-Secure-App-WAF |
+-----+
tmp $ export WAF_ARN=$(aws wafv2 list-web-acls \
> --scope REGIONAL \
> --region ap-south-1 \
> --query "WebACLS[?Name=='SG-Secure-App-WAF'].ARN" | [0]) \
> --output text)
tmp $
tmp $ echo "$WAF_ARN"
arn:aws:wafv2:ap-south-1:547641909662:regional/webacl/SG-Secure-App-WAF/c0fc6332-b606-4c21-a32e-63edf70a110c
tmp $ LOG_ARN=$(aws logs describe-log-groups \
> --region ap-south-1 \
> --log-group-name-prefix aws-waf-logs-SG-Secure-App-WAF \
> --query 'logGroups[0].arn' \
> --output text)
tmp $
tmp $ echo "$LOG_ARN"
arn:aws:logs:ap-south-1:547641909662:log-group:aws-waf-logs-SG-Secure-App-WAF:*
tmp $ aws wafv2 put-logging-configuration \
> --logging-configuration "ResourceArn=$WAF_ARN,LogDestinationConfigs=$LOG_ARN" \
> --region ap-south-1
{
  "LoggingConfiguration": {
    "ResourceArn": "arn:aws:wafv2:ap-south-1:547641909662:regional/webacl/SG-Secure-App-WAF/c0fc6332-b606-4c21-a32e-63edf70a110c",
    "LogDestinationConfigs": [
      {
        "arn:aws:logs:ap-south-1:547641909662:log-group:aws-waf-logs-SG-Secure-App-WAF:*"
      }
    ],
    "ManagedByFirewallManager": false,
    "LogType": "WAF_LOGS",
    "LogScope": "CUSTOMER"
  }
}
tmp $
```

Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates

The project-role Access Analyzer review produced no matching external-access finding for SG-Secure-App resources. Combined with the scoped policy and simulation evidence, this supports the conclusion that the project roles were deliberately constrained and did not require long-lived application credentials.

## Phase 5 — Audit + Layered Attack Walkthrough

The audit layer combines CloudTrail, WAF CloudWatch Logs, metric filters and CloudWatch alarms. The attack walkthrough maps anonymous access, WAF-triggering input, non-admin authorization and unauthorized cryptographic access to the layer that stops each event.

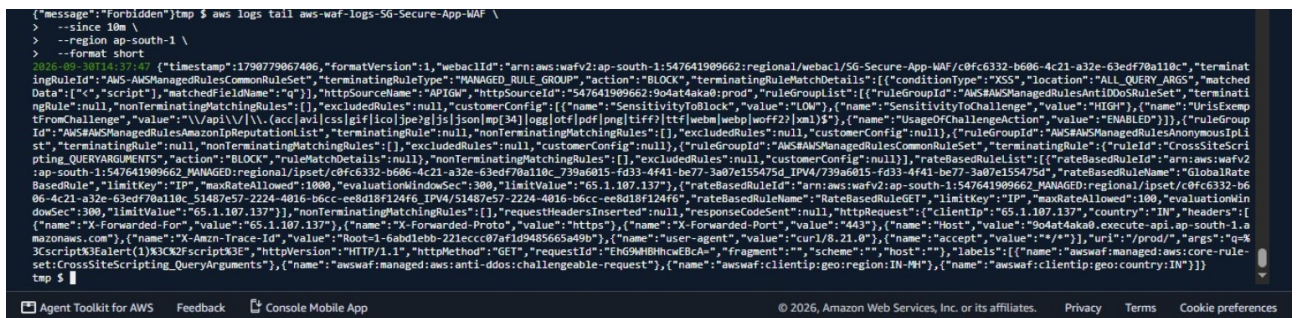
Outcome: WAF logs, CloudTrail events and CloudWatch alarms formed an auditable detection chain, including a triggered denied-decrypt alarm.

**Figure 90: Evidence screenshot**



The XSS-like API request again returns HTTP 403 Forbidden. In the Phase 5 walkthrough this represents the WAF-triggering attack step: the edge layer rejects the request before the protected API path can process it normally.

**Figure 91: Evidence screenshot**



The detailed CloudTrail/KMS evidence shows a Decrypt event with AccessDenied for the unauthorized test role. The event contains the KMS service, action, caller identity and denial message, making the negative cryptographic test independently auditable.

Figure 92: Evidence screenshot

```
tmp $ aws lambda invoke \
> --function-name SG-Secure-App-KMS-Unauthorized-Test \
> --payload fileb:///tmp/phase5-kms-denied.json \
> --region ap-south-1 \
> /tmp/phase5-kms-result.json
{
  "StatusCode": 200,
  "FunctionError": "Unhandled",
  "ExecutedVersion": "$LATEST"
}
tmp $
tmp $ cat /tmp/phase5-kms-result.json
{"errorMessage": "An error occurred (AccessDeniedException) when calling the Decrypt operation: User: arn:aws:sts::547641909662:assumed-role/SG-Secure-App-KMS-Unautho
rized-Test is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-south-1:547641909662:key/a44dc72b-1465-4b0e-8253-5444d1573ac5 because no identity-based policy allows the kms:Dec
rypt action", "errorType": "ClientError", "requestId": "1c73ba5d-f9c4-4de1-81f2-53dee71ef08", "stackTrace": [ " File \"/var/task/kms_denied_lambda.py", line 9, in lambda_handler\n response =
kms.decrypt(\n, " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 606, in _api_call\n return self._make_api_call(operation_name, kwargs)\n", " File \"/var/lang/lib/p
ython3.12/site-packages/botocore/context.py", line 123, in wrapper\n return func(*args, **kwargs)\n", " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 1894, in _make
_api_call\n raise error_class(parsed_response, operation_name)\n"]}]tmp $
```

The WAF logging configuration points SG-Secure-App-WAF to the CloudWatch Logs destination `aws-waf-logs-SG-Secure-App-WAF`. This establishes the raw WAF audit stream required for the attack walkthrough and operational investigation.

Figure 93: Evidence screenshot

```
> --log-group-name /aws/cloudtrail/SG-Secure-App-Audit \
> --filter-pattern '{ ($eventName = "Decrypt") && ($errorCode = "AccessDenied") }' \
> --region ap-south-1 \
> --query 'events[0].message' \
> --output text
{"eventVersion": "1.11", "userIdentity": {"type": "AssumedRole", "principalId": "AROAX7AQAMGPBG0A6FAVZ:SG-Secure-App-KMS-Unauthorized-Test", "arn": "arn:aws:sts::547641909662:assumed-role/SG-Secure-App-Un
authorized-Role/SG-Secure-App-KMS-Unauthorized-Test", "accountId": "547641909662", "accessKeyId": "ASIA7AQAMGPBG0A6FAVZ", "sessionContext": {"sessionIssuer": {"type": "Role", "principalId": "AROAX7AQAMGPBG0
A6FAVZ", "arn": "arn:aws:iam::547641909662:role/SG-Secure-App-KMS-Unauthorized-Role", "accountId": "547641909662", "userName": "SG-Secure-App-KMS-Unauthorized-Role", "attributes": {"creationDate": "2026-09-30T15:
06:23Z", "mfaAuthenticated": "false"}}, "inScopeOf": {"issuerType": "AWS::Lambda::Function", "credentialsIssuedTo": "arn:aws:lambda:ap-south-1:547641909662:function:SG-Secure-App-KMS-Unauthorized-Test", "
credentialsIssuedToVersion": "arn:aws:lambda:ap-south-1:547641909662:function:SG-Secure-App-KMS-Unauthorized-Test:$LATEST"}}, "eventTime": "2026-09-30T15:14:58Z", "eventSource": "kms.amazonaws.com", "ev
entName": "Decrypt", "awsRegion": "ap-south-1", "sourceIPAddress": "65.1.110.188", "userAgent": "Boto3/1.42.97 md/Botocore/1.42.97 ua/2.1 os/linux5.10.255-268-309.1892.amzn2.x86_64 md/arch#x86_64 lang/p
ython3.12.14 md/pymp1CPython exec-env/AWS_Lambda_python3.12 m/2,g.D.b c/fg/retry-modeLegacy Botocore/1.42.97", "errorCode": "AccessDenied", "errorMessage": "User: arn:aws:sts::547641909662:assumed-
role/SG-Secure-App-KMS-Unauthorized-Role/SG-Secure-App-KMS-Unauthorized-Test is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-south-1:547641909662:key/a44dc72b-1465-4b0e-8253-5444
d1573ac5 because no identity-based policy allows the kms:Decrypt action", "requestParameters": null, "responseElements": null, "requestID": "16fca9d4-8983-4fa7-bfdd-210776e11ccb", "eventID": "feb1695f-88a
1-4abc-a475-de3d4183ae83", "readOnly": true, "eventType": "AwsApiCall", "managementEvent": true, "recipientAccountId": "547641909662", "eventCategory": "Management", "tlsDetails": {"tlsVersion": "TLSv1.3", "ci
pherSuite": "TLS_AES_256_GCM_SHA384", "clientProvidedHostHeader": "kms.ap-south-1.amazonaws.com", "keyExchange": "x25519"}}
tmp $
```

The CloudWatch log-stream query shows an active WAF log stream named `ap-south-1_SG-Secure-App-WAF_0` with a recent last-event timestamp. This confirms that WAF logging is not merely configured; an actual log stream is receiving events.

Figure 94: Evidence screenshot

```
tmp $ aws cloudwatch describe-alarms \
> --alarm-names "SG-Secure-App-WAF-Blocked-Spike-Alarm" \
> --region ap-south-1 \
> --query 'MetricAlarms[].[Name:AlarmName,State:StateValue,Metric:MetricName,Threshold:Threshold]' \
> --output table
```

| DescribeAlarms  |                                       |                   |           |
|-----------------|---------------------------------------|-------------------|-----------|
| Metric          | Name                                  | State             | Threshold |
| BlockedRequests | SG-Secure-App-WAF-Blocked-Spike-Alarm | INSUFFICIENT_DATA | 10.0      |

```
tmp $
```

Agent Toolkit for AWS   Feedback   Console Mobile App   © 2026, Amazon Web Services, Inc. or its affiliates

The WAF log event records action=BLOCK and identifies AWS-AWSManagedRulesCommonRuleSet as the terminating managed rule group. The matched query contains the XSS-like script input, providing strong evidence that the WAF blocked the malicious request and recorded the reason.

Figure 95: Evidence screenshot

```
tmp $ aws cloudwatch describe-alarms \
> --alarm-names "SG-Secure-App-Failed-Signin-Alarm" \
> --region ap-south-1 \
> --query 'MetricAlarms[].[Name:AlarmName,State:StateValue,Metric:MetricName,Threshold:Threshold]' \
> --output table
```

| DescribeAlarms    |                                   |       |           |
|-------------------|-----------------------------------|-------|-----------|
| Metric            | Name                              | State | Threshold |
| FailedSigninCount | SG-Secure-App-Failed-Signin-Alarm | OK    | 1.0       |

```
tmp $
```

Agent Toolkit for AWS   Feedback   Console Mobile App   © 2026, Amazon Web S

The corresponding API request returns HTTP 403 Forbidden for the XSS-like query. The request timestamp aligns with the WAF log event, linking the client-visible block to the security telemetry generated by the web ACL.

Figure 96: Evidence screenshot

```
> --filter-pattern '({$.eventSource = "kms.amazonaws.com"}) && ($.eventName = "Decrypt") && ($.errorCode = "AccessDenied") && ($.userIdentity.sessionContext.sessionIssuer.userName = "SG-Secure-App-Unauthenticated-Role")' \
> --region ap-south-1 \
> --query 'events[0].message' \
> --output text
{"eventVersion":"1.11","userIdentity":{"type":"AssumedRole","principalId":"AROAX7AQWGP8G6AFVZ:SG-Secure-App-KMS-Unauthenticated-Test","arn":"arn:aws:sts:547641909662:assumed-role/SG-Secure-App-Unauthenticated-Role/SG-Secure-App-KMS-Unauthenticated-Test","accountId":"547641909662","accessKeyId":"ASIAIAX7AQWGP8G6AFVZ","sessionContext":{"sessionIssuer":{"type":"Role","principalId":"AROAX7AQWGP8G6AFVZ","arn":"arn:aws:iam:547641909662:role/SG-Secure-App-Unauthenticated-Role","accountId":"547641909662","userName":"SG-Secure-App-Unauthenticated-Role"},"attributes":{"creationDate":"2026-09-30T15:06:23Z","mfaAuthenticated":"false"},"inscopeOf":{"issuerType":"AWS::Lambda::Function","credentialsIssuedTo":"arn:aws:lambda:ap-south-1:547641909662:function:SG-Secure-App-KMS-Unauthenticated-Test","credentialsIssuedToVersion":"arn:aws:lambda:ap-south-1:547641909662:function:SG-Secure-App-KMS-Unauthenticated-Test:$LATEST"}}},"eventTime":"2026-09-30T15:14:58Z","eventSource":"kms.amazonaws.com","eventName":"Decrypt","awsRegion":"ap-south-1","sourceIPAddress":"65.1.110.188","userAgent":"Boto3/1.42.97 md/BotoCore/1.42.97 ua/2.1 os/linux#5.10.255-268-309.1092.amzn2.x86_64 md/arch#x86_64 lang/python3.12.14 md/python3.12.14 python exec-env/AWS_Lambda.python3.12.12/gD.b cfa/retry-mode#legacy BotoCore/1.42.97","errorCode":"AccessDenied","errorMessage":"User: arn:aws:sts:547641909662:assumed-role/SG-Secure-App-Unauthenticated-Role/SG-Secure-App-KMS-Unauthenticated-Test is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-south-1:547641909662:key/a44dc72b-1465-4b0e-8253-5444d1573ac5 because no identity-based policy allows the kms:Decrypt action","requestParameters":null,"responseElements":null,"requestID":"16fca9d4-8983-4fa7-bfdd-210776e11ccb","eventID":"feb1695f-08a1-4abc-a475-de3d4183ae83","readOnly":true,"eventType":"AwsApiCall","managementEvent":true,"recipientAccountId":"547641909662","eventCategory":"Management","tlsDetails":{"tlsVersion":"TLSv1.3","cipherSuite":"TLS_AES_256_GCM_SHA384","clientProvidedHostHeader":"kms.ap-south-1.amazonaws.com","keyExchange":"X25519"}}
tmp $
```

The KMS negative test again produces AccessDeniedException for kms:Decrypt. Repeating the test provides an additional independent evidence point that the cryptographic boundary consistently rejects the unauthorized role.

Figure 97: Evidence screenshot

```
tmp $ CT=$(aws kms encrypt \
> --key-id alias/alias/SG-Secure-App-Secrets-Key \
> --plaintext 'Phase5-KMS-Test' \
> --cli-binary-format raw-in-base64-out \
> --region ap-south-1 \
> --query CiphertextBlob \
> --output text)
tmp $
tmp $ printf '{"ciphertext":"%s"}' "$CT" > /tmp/phase5-kms-denied.json
tmp $ cat /tmp/phase5-kms-denied.json
{"ciphertext":"AQICAHi+3BmTgP8Ewi3UePY+aFyS71R1DhrAu8cqx36gOV11/wHLDfQNDt5SXQaE0y3ZYkFRAAAABTB8rkqhki69w0B8WagXjBcAgEAMF cGCSqGSIb3DQEHAEEBg1ghkgB8ZQMES4wEQQM79Ms712/VegDFG3NAgEQCrpZSzAgjcsjXICm
fig3lOaAc/ZkhtcXOMPAm4nErcpxhox25jpnV8G7Hg="}tmp $ aws lambda invoke \
> --function-name SG-Secure-App-KMS-Unauthenticated-Test \
> --payload file:///tmp/phase5-kms-denied.json \
> --region ap-south-1 \
> /tmp/phase5-denied-retest.json
{
  "StatusCode": 200,
  "FunctionError": "Unhandled",
  "ExecutedVersion": "$LATEST"
}
tmp $ cat /tmp/phase5-denied-retest.json
{"errorMessage":"An error occurred (AccessDeniedException) when calling the Decrypt operation: User: arn:aws:sts:547641909662:assumed-role/SG-Secure-App-Unauthenticated-Role/SG-Secure-App-KMS-Unauthenticated-Test is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-south-1:547641909662:key/a44dc72b-1465-4b0e-8253-5444d1573ac5 because no identity-based policy allows the kms:Decrypt action","errorType":"ClientError","requestId":"b45ee8ea-9c74-49d6-bd18-790f74c247ae","stackTrace":[" File \"/var/task/kms_denied_lambda.py", line 9, in lambda_handler\n response = kms.decrypt(Vn, " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 606, in _api_call\n return self._make_api_call(operation_name, kwargs)\n", " File \"/var/lang/lib/python3.12/site-packages/botocore/context.py", line 123, in wrapper\n return func(*args, **kwargs)\n", " File \"/var/lang/lib/python3.12/site-packages/botocore/client.py", line 1094, in _make_api_call\n raise error_class(parsed_response, operation_name)\n"]}]tmp $
```

The CloudWatch alarm configuration shows the FailedSignInCount alarm in OK state with threshold 1. The alarm is configured to watch the failed-sign-in metric; OK indicates that no threshold breach was active in the displayed evaluation period.

Figure 98: Evidence screenshot

```
tmp $ aws cloudwatch describe-alarms \
> --alarm-names \
> "SG-Secure-App-Denied-Decrypt-Alarm" \
> "SG-Secure-App-WAF-Blocked-Spike-Alarm" \
> "SG-Secure-App-Failed-Signin-Alarm" \
> --region ap-south-1 \
> --query 'MetricAlarms[].[Name:AlarmName,State:StateValue,Metric:MetricName,Threshold:Threshold]' \
> --output table
```

| Metric             | Name                                  | State | Threshold |
|--------------------|---------------------------------------|-------|-----------|
| DeniedDecryptCount | SG-Secure-App-Denied-Decrypt-Alarm    | ALARM | 1.0       |
| FailedSignInCount  | SG-Secure-App-Failed-Signin-Alarm     | OK    | 1.0       |
| BlockedRequests    | SG-Secure-App-WAF-Blocked-Spike-Alarm | OK    | 10.0      |

```
tmp $ aws wafv2 list-web-acls \
> --scope REGIONAL \
> --region ap-south-1 \
> --query "WebACLS[?Name=='SG-Secure-App-WAF'].{Name:Name,Id:Id,ARN:ARN}" \
> --output table
```

| ARN                                                                                                          | Id                                   | Name              |
|--------------------------------------------------------------------------------------------------------------|--------------------------------------|-------------------|
| arn:aws:wafv2:ap-south-1:547641909662:regional/webacl/SG-Secure-App-WAF/c0fc6332-b606-4c21-a32e-63edf70a110c | c0fc6332-b606-4c21-a32e-63edf70a110c | SG-Secure-App-WAF |

The WAF blocked-spike alarm is configured on the BlockedRequests metric with a threshold of 10. The displayed INSUFFICIENT\_DATA state is a valid alarm state when a metric has not yet produced enough recent datapoints for evaluation; the configuration itself is present.

Figure 99: Evidence screenshot

```
arn:aws:wafv2:ap-south-1:547641909662:regional/webacl/SG-Secure-App-WAF/c0fc6332-b606-4c21-a32e-63edf70a110c | c0fc6332-b606-4c21-a32e-63edf70a110c | SG-Sec
```

```
tmp $ WEB_ACL_ARN=$(aws wafv2 list-web-acls --scope REGIONAL --region ap-south-1 --query "WebACLS[?Name=='SG-Secure-App-WAF'].ARN | [0]" --output text)
tmp $
tmp $ aws wafv2 get-logging-configuration \
> --resource-arn "$WEB_ACL_ARN" \
> --region ap-south-1
```

```
{
  "LoggingConfiguration": {
    "ResourceArn": "arn:aws:wafv2:ap-south-1:547641909662:regional/webacl/SG-Secure-App-WAF/c0fc6332-b606-4c21-a32e-63edf70a110c",
    "LogDestinationConfigs": [
      "arn:aws:logs:ap-south-1:547641909662:log-group:aws-waf-logs-SG-Secure-App-WAF:*"
    ],
    "ManagedByFirewallManager": false,
    "LogType": "WAF_LOGS",
    "LogScope": "CUSTOMER"
  }
}
```

```
tmp $ aws logs describe-log-streams \
> --log-group-name "aws-waf-logs-SG-Secure-App-WAF" \
> --region ap-south-1 \
> --order-by LastEventTime \
> --descending \
> --query 'logStreams[].[Stream:logStreamName,LastEvent:lastEventTimestamp]' \
> --output table
```

| LastEvent     | Stream                         |
|---------------|--------------------------------|
| 1790779067406 | ap-south-1_SG-Secure-App-WAF_0 |

The combined CloudWatch alarm view shows three security alarms: DeniedDecryptCount in ALARM, FailedSignInCount in OK, and BlockedRequests in OK. The denied-decrypt alarm demonstrates a triggered security signal, while the other two remain configured and ready to detect their respective conditions.



## Phase 6 — Report, Dry Run and Teardown

Phase 6 is primarily the documentation and operational-readiness stage. The supplied guide requires a README and security/architecture diagram, a report covering the layers and independent proofs, a live attack walkthrough, identification of the weakest remaining link, a dry run of the demo, and cleanup of cloud resources.

### 6.1 Layer-by-Layer Independent Proof

| Layer             | Independent proof                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Identity          | Cognito MFA configuration and successful TOTP authentication; 401 without a bearer token; 200 for an authenticated user            |
| Edge              | WAF association and rule set; legitimate API request returns 200; XSS-like request returns 403; WAF log records action             |
| Data              | Customer-managed KMS key; Secrets Manager rotation state; runtime secret retrieval; unauthorized GetSecretValue denied             |
| Least privilege   | Scoped runtime/rotation policies; allowed secret access; implicit deny for unrelated S3 action; implicit deny for another resource |
| Audit / detection | CloudTrail GetSecretValue/Decrypt events; WAF CloudWatch log stream and block event; CloudWatch alarms; Deny events                |

### 6.2 Layered Attack Walkthrough

**Step 1 — Anonymous:** request the protected `/api/user` route without a bearer token → HTTP 401.

**Step 2 — WAF-triggering:** send the XSS-like query to the WAF-protected API → HTTP 403; CloudWatch WAF log shows BLOCK and the terminating managed rule.

**Step 3 — Authenticated non-admin:** use a valid user access token against `/api/admin` → HTTP 403 because admin-group membership is required.

**Step 4 — Scoped component:** invoke the runtime Lambda → secret retrieval succeeds; invoke the unauthorized test role → GetSecretValue or KMS Decrypt is denied and auditable in CloudTrail.

**Step 5 — Detection:** review CloudTrail/WAF logs and CloudWatch alarms to answer who accessed what, when, and which control stopped the request.

### 6.3 Weakest Remaining Link / Residual Risk

The principal remaining architectural limitation is that the evidence proves the Cognito/JWT local API path and the WAF/API Gateway path independently rather than as one fully integrated production request chain. A second limitation is that the custom rotation Lambda's `setSecret` step is intentionally a no-op because the project uses a test API key without a real downstream credentialed service. For a production deployment, the next improvement would be to place the authenticated application behind the same WAF-protected API front door and connect rotation to the real credential consumer.

### 6.4 Dry-Run Checklist

| Demo step | Expected evidence                                                                                         |
|-----------|-----------------------------------------------------------------------------------------------------------|
| 1         | Show Cognito user pool, MFA/TOTP and admin/user groups.                                                   |
| 2         | Run anonymous request → 401.                                                                              |
| 3         | Run valid user request → 200; run user against admin route → 403; run admin route with admin token → 200. |
| 4         | Show WAF rules and send legitimate request → 200.                                                         |
| 5         | Send XSS-like request → 403 and show WAF log with terminating rule.                                       |
| 6         | Show KMS key, secret rotation state and runtime secret retrieval.                                         |

| Demo step | Expected evidence                                                       |
|-----------|-------------------------------------------------------------------------|
| 7         | Show unauthorized secret/KMS denial and corresponding CloudTrail event. |
| 8         | Show IAM policy scope, simulations and Access Analyzer.                 |
| 9         | Show CloudWatch alarms, including the triggered denied-decrypt signal.  |
| 10        | Answer 'who accessed what, when' using CloudTrail/WAF logs.             |

## 6.5 Cleanup / Teardown Checklist

After the live review, delete or disable temporary test resources according to the assignment's teardown requirement: test Lambdas and their execution roles, temporary API Gateway/WAF resources, CloudWatch alarms and test log groups where no longer required, the dedicated test secret, test Cognito users/pool, and the customer-managed KMS key only after confirming no retained ciphertext depends on it. Preserve the final report and evidence before deletion. Do not delete shared/company resources or unrelated findings.

## 6.6 GitHub README Deliverable

The GitHub repository contains the project README, application code, supporting documentation and the security/architecture diagram. The final project report is maintained as a separate PDF deliverable in the repository.

# Final Summary

The Secure Application Platform demonstrates a layered AWS security design in which authentication, authorization, edge filtering, secret protection, cryptographic access control, least-privilege IAM and audit monitoring are independently tested. The evidence shows Cognito MFA and group authorization, WAF blocking and rate controls, customer-managed KMS protection, Secrets Manager rotation and runtime retrieval, explicit denial of unauthorized secret/KMS operations, scoped IAM policies, Access Analyzer review, CloudTrail audit records and CloudWatch security alarms.

The project therefore satisfies the core technical proof model of the capstone: each major security layer has an independent positive or negative test and an evidence trail. The remaining risks are documented rather than hidden, especially the separation between the WAF/API Gateway test path and the local Cognito/JWT application path, and the test-only nature of the rotation consumer. These are the primary areas to address when evolving the demonstration into a single production-style deployment.

## Research

Practical research was performed independently while building and validating the project in the AWS console and CloudShell. The following AWS documentation was used as the primary technical reference set:

| Source                                | Research use                                                                                               |
|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| Amazon Cognito documentation          | MFA/TOTP configuration, managed login and access-token/group authorization concepts.                       |
| AWS WAF documentation                 | Managed rule groups, rate-based request limiting and rule evaluation behavior.                             |
| AWS Secrets Manager documentation     | Lambda rotation functions, four-step rotation lifecycle, customer-managed KMS encryption and rotation      |
| AWS IAM Access Analyzer documentation | Access findings, least-privilege review and analysis of unintended/unused access.                          |
| AWS CloudTrail documentation          | Event history and audit-event interpretation for API activity, including identity, action and resource con |

Primary references: AWS Cognito TOTP software-token MFA and MFA configuration documentation; AWS WAF rate-based rule documentation; AWS Secrets Manager Lambda rotation and encryption documentation; AWS IAM Access Analyzer documentation; and AWS CloudTrail Event History documentation.

End of Report